ARE SELF-EVOLVING AGENT SKILLS LEARNING SKILLS?

Anonymous Submission

ABSTRACT

Self-evolving agent skills turn execution experience into persistent instructions and code, but improved performance on the source task leaves open what has been learned. We study this question through repeated standard-task evaluations, controlled task variations, and inspection of the resulting skills and their use. Our diagnosis distinguishes source-task utility from usefulness under changes to input representation, task instances, and requirements. We examine SkillEvolver, SPARK, and CoEvoSkills, connecting their observed reuse patterns to the decision rules preserved in their skills and the feedback used during skill construction. The results reveal different scopes of generalization despite positive source-task gains, and motivate evaluating skills by how they guide decisions when the task changes.

***Index Terms*—** Agent skills, self-evolution, diagnostic evaluation, skill reuse, generalization

1. INTRODUCTION

Persistent agent skills store instructions, scripts, and references that an executor can reuse across tasks. Skill-generation and evolution methods turn execution experience into these external artifacts without changing the executor's model parameters. SkillEvolver refines skills from their deployment outcomes, SPARK distills environment-verified trajectories, and CoEvoSkills jointly evolves skills and a surrogate verifier [1–3]. Their common goal is to make experience useful beyond the attempt that produced it.

Success on the source task does not reveal the scope of that usefulness. A skill can help by encoding a procedure that adapts to new inputs, or by preserving choices that happen to work on one instance. Both can raise a benchmark score. The distinction becomes visible when the executor must reinterpret an input, solve a replacement instance, or satisfy an additional requirement while using the same skill. These changes test what the artifact preserves from experience and what the executor must still infer.

We investigate this distinction as a diagnostic study of skill reuse. We first measure each method's gain over a same-task no-skill baseline in two standard-task runs. We then freeze the skills and examine their utility under changes to representation, instances, and requirements. Finally, we inspect contrasting examples to connect successful and unsuccessful reuse to the guidance expressed in the skills. This sequence moves from whether a skill helps to where it helps and what its contents suggest about the difference.

The resulting picture differs across methods. All three show positive paired gains in both standard-task runs. Under task variation, SkillEvolver retains positive average gains across all three change types, SPARK's gains depend on the type of change, and CoEvoSkills' average gains reverse sign. Instance changes produce the clearest separation. Thus recurring source-task gains and generalization describe different properties of an evolved skill.

The case evidence makes this distinction concrete. In signal detection, selecting a strong candidate is insufficient when the signal can be confused with background variability or a harmonic. In scheduling, a feasible output can still violate the task's repair rules. Successful reuse in these examples accompanies skills that preserve criteria for choosing and checking a result. We interpret these differences in light of the methods' feedback and skill-construction procedures.

This paper contributes a diagnosis of skill learning through three complementary forms of evidence:

- **Repeated utility measurement.** Six standard-task runs establish the observed paired benefits of SkillEvolver, SPARK, and CoEvoSkills on their evaluated tasks.
- **Task-variation analysis.** Representation, instance, and requirement changes expose different scopes of frozen-skill usefulness across the recorded variant evaluations.
- **Skill-level interpretation.** Contrasting reuse examples connect outcomes to preserved decision criteria and motivate an explanation based on feedback coverage and abstraction from the source instance.

2. PROBLEM FORMULATION

2.1. Skill Revision and Evaluation

At revision round r, an optimizer updates the current persistent skill artifact:

$$s_{r+1} = O(s_r, \tau_r, f_r), \tag{1}$$

where s_r is the current skill, τ_r is an execution trace, and f_r is feedback from execution and evaluation. The operator O denotes the method's complete update pathway, including its native merge, validation, selection, or adoption mechanism. A generated candidate that is rejected or never adopted is not s_{r+1}.

The agent then executes task x with the frozen skill:

$$o_r = A(x, s_r, z_r), \tag{2}$$

where z_r represents model sampling, tool use, and environment state, and o_r is the resulting output artifact or state change. A verifier maps the output to a score:

$$y_r = V(x, o_r). \tag{3}$$

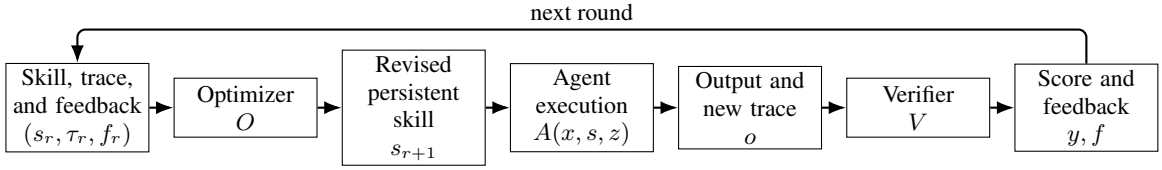


Fig. 1. Skill evolution and deployment. Our evaluations examine the utility of the completed skill and its reuse under task changes; skill contents and execution traces provide evidence for interpreting the observed behavior.

2.2. Diagnosing What a Skill Has Learned

Let s_x denote the completed skill for source task x, and let $T_k(x)$ denote a task variant. The skill remains fixed when it is used on $T_k(x)$. We compare skill-assisted and no-skill executions of the same task or variant using its verifier. A positive paired difference indicates utility in that setting. Its persistence across changes indicates the observed scope of reuse.

We distinguish three questions. First, do positive source-task gains recur across runs? Second, how does the same skill perform when the representation, instance, or requirements change? Third, which guidance in the skill is associated with successful or failed reuse? The first two questions are answered by paired outcomes. The third is examined through skill contents, execution behavior, and the published construction procedures. The executor can adapt its actions on a variant, but the skill itself is not revised during this evaluation.

3. DIAGNOSING THE SCOPE OF SKILL UTILITY

We examine paired utility on standard tasks before evaluating frozen skills under three forms of task variation. The comparison establishes where the completed skills help and which changes alter their usefulness.

3.1. Experimental Setup

We evaluate SkillEvolver [1], SPARK [2], and CoEvoSkills [3] on SkillsBench [4], with two runs per method. Each run pairs the method's skill condition with a no-skill execution of the same task. Both runs of each method use the same task set: 81 tasks for SkillEvolver, 84 for SPARK, and 85 for CoEvoSkills.

We report mean verifier reward on a 0–100 scale and the mean within-task difference between the skill and no-skill conditions. For a run with N tasks, the paired gain is

$$\Delta = \frac{100}{N} \sum_{i=1}^{N} \left(r_i^{\text{skill}} - r_i^{\text{no-skill}} \right). \tag{4}$$

We also count tasks with positive, negative, and zero differences.

3.2. Source-Task Gains Recur across Runs

All three methods have positive mean paired gains in both runs (Table 1). SkillEvolver gains 25.61 and 25.02 points, CoEvoSkills gains 10.24 and 9.76, and SPARK gains 1.84 and 6.45. SkillEvolver and CoEvoSkills retain similar gains despite substantial changes in their no-skill scores. Their benefit is therefore more consistent across runs than their absolute skill-condition scores suggest.

The task-level results also differ. SkillEvolver has at least four times as many improvements as regressions in each run; CoEvoSkills improves 17 tasks in each run, with nine and ten regressions. SPARK's first run is nearly balanced, while its second has 13 improvements and five regressions. These results establish recurring source-task utility. We next ask whether the same frozen skills remain useful after the task changes.

Method	Run	Tasks	No skill	Skill	Δ	Improved	Regressed	Tied
SkillEvolver	1	81	30.20	55.81	+25.61	27	6	48
SkillEvolver	2	81	37.32	62.34	+25.02	26	3	52
SPARK	1	84	17.82	19.66	+1.84	9	8	67
SPARK	2	84	14.37	20.81	+6.45	13	5	66
CoEvoSkills	1	85	42.21	52.44	+10.24	17	9	59
CoEvoSkills	2	85	23.54	33.31	+9.76	17	10	58

Table 1. Two runs per method on SkillsBench. Scores are mean verifier rewards multiplied by 100; Δ is the mean paired gain in percentage points. Each method uses an identical task set across its two runs.

3.3. Task Changes Separate Different Scopes of Utility

We apply each source-task skill to controlled variants without further evolution. Input-representation changes (P1) alter names, column arrangements, or schemas; task-instance changes (P2) replace the data or instance structure; coupled-requirement changes (P3) alter or add requirements that must be satisfied together. These variations test whether a frozen skill remains useful when the executor must reinterpret inputs, recompute a solution, or accommodate new requirements.

The complete record contains 222 variants and 886 condition-level results across no skill and the three methods. Table 2 summarizes historical and recovery evaluations for 222 SkillEvolver variants and 221 each for SPARK and CoEvoSkills, pairing each skill condition with the no-skill result on the same variant.

Source-task improvement does not imply useful generalization. All three methods improve mean reward on the source tasks in both runs, but their variant results diverge. SkillEvolver retains a positive mean gain of 11.94 points, SPARK is close to neutral at $+0.65$, and CoEvoSkills falls below the no-skill baseline at -5.01. SkillEvolver also improves more variants than it regresses (47 versus 20), whereas

SPARK and CoEvoSkills show the reverse. Thus, the source-task gains of the latter two methods do not translate into a consistent advantage on changed tasks.

Replacing the instance produces the largest divergence. SkillEvolver remains positive under all three changes, with its largest gain on P2 ($+14.56$). P2 is also where SPARK and CoEvoSkills decline most (-5.23 and -9.51). SPARK retains positive averages under representation and requirement changes, but loses its advantage when the underlying instance changes. SkillEvolver's positive results across P1–P3 indicate a broader observed scope of skill utility. CoEvoSkills' negative averages across all three changes show that its source-task benefit is particularly sensitive to reuse conditions.

A. Overall utility on task variants

Method	Variants	No skill	Skill	Δ	Improved	Regressed	Tied
SkillEvolver	222	41.87	53.81	$+11.94$	47	20	155
SPARK	221	41.61	42.25	$+0.65$	32	37	152
CoEvoSkills	221	41.61	36.60	-5.01	26	41	154

B. Utility by type of task change

Method	P1: Representation		P2: Instance		P3: Requirements	
	N	Δ	N	Δ	N	Δ
SkillEvolver	77	$+11.85$	68	$+14.56$	77	$+9.72$
SPARK	77	$+2.00$	68	-5.23	76	$+4.53$
CoEvoSkills	77	-3.59	68	-9.51	76	-2.42

Table 2. Skill utility under task variation: descriptive summary of historical and recovery evaluations. Panel A reports overall outcomes; Panel B separates representation, instance, and requirement changes. Each method is paired with its own same-variant no-skill baseline. Scores and within-variant differences use the scale in Table 1.

4. EXPLAINING THE OBSERVED SCOPE OF REUSE

The variant results locate differences in skill utility. We examine two task families to make those differences interpretable: signal detection, where the executor must choose between plausible candidates, and scheduling, where the output must satisfy specific repair rules. These cases connect observed outcomes to the guidance preserved in the frozen skills.

4.1. Distinguishing a Candidate from a Valid Answer

In exoplanet detection, the SkillEvolver skill supports successful detection after input columns are rearranged (P1) and after the task requires two signals (P3). The CoEvoSkills skill fails both variants: its use leads to stellar modulation being reported as the target signal, and a signal and its harmonic being treated as two separate signals. Both skills fail the replacement-instance P2 variant.

The difference is visible in what the skills ask the executor to check. SkillEvolver separates candidate discovery, signal diagnosis, and refinement. Its guidance addresses misleading peaks, transit-like durations, and harmonic relationships. CoEvoSkills also includes validation, including phase-fold, odd/even, and residual checks, but its use in these variants does not resolve the misleading candidates. The presence of a checking routine is therefore less informative than whether its criteria distinguish the alternatives that matter on the changed task. SkillEvolver's successful P1 and P3 deployments illustrate that distinction, while its P2 failure shows that those criteria do not solve every new instance.

4.2. Preserving the Rules That Define a Correct Repair

In scheduling, the SkillEvolver and SPARK skills succeed on all three variants; the CoEvoSkills skill succeeds only on P3. In P1, using CoEvoSkills yields an otherwise feasible schedule that fails a required local-minimality check on the baseline-relative repair. Feasibility checks alone do not capture this local-minimality requirement.

The SkillEvolver skill explicitly retains local minimality and baseline-relative repair rules. These rules specify how an acceptable repair differs from an arbitrary feasible schedule, and remain relevant after the input names or scheduling instance change. The CoEvoSkills skill covers several structural and feasibility checks, yet the failed deployment misses the decisive repair condition. The contrast concerns which correctness conditions survive in the reusable guidance. SPARK's success across the same three variants also shows that this form of successful reuse is not exclusive to one method.

4.3. Construction Feedback and Skill Content

The construction procedures offer an interpretation of these skill-level differences. SkillEvolver compares successful and failed deployments of a candidate skill and patches missing or misleading guidance. Its independent audit also checks reliance on training-instance constants [1]. This exposes failures in using the skill and encourages rules that a later executor can apply to a different input. The scheduling and signal-detection skills reflect that emphasis by retaining criteria for checking a repair or discriminating between candidate signals.

SPARK grounds skill distillation in environment-verified trajectories, preserving execution evidence, checks, and lessons [2]. Its scheduling result shows that such evidence can support successful reuse. Its mixed aggregate profile raises a separate question about abstraction: a verified execution supplies evidence for what worked on the source task, while changed instances require determining which choices should be retained and which should be recomputed. The observed decline on P2 makes that distinction especially relevant to interpreting its results.

CoEvoSkills obtains detailed revision feedback from a co-evolving surrogate verifier, while the ground-truth oracle returns an opaque pass/fail signal [3]. This makes the surrogate's coverage of correctness conditions relevant to the guidance retained in the skill. This provides a plausible account of the scheduling failure: the skill preserves many checks, but successful reuse still depends on a condition those checks did not enforce in that deployment. Across the cases, feedback coverage and abstraction from the source instance are candidate explanations for the different scopes of utility.

5. IMPLICATIONS FOR REUSABLE SKILL LEARNING

5.1. Average Gain and Improvement Frequency

SPARK illustrates a distinction between the size and frequency of benefits. Its overall variant mean is slightly positive even though regressions outnumber improvements. A near-zero aggregate therefore does not mean that the skill has no effect: beneficial and harmful changes offset one another. SkillEvolver's positive mean is accompanied by more improvements than regressions, whereas CoEvoSkills has both a negative mean and more regressions. These outcome profiles describe whether a skill's aggregate utility reflects broad benefit among changed outcomes or a balance of opposing changes.

5.2. Implications for Skill Construction and Evaluation

The cases suggest a concrete target for skill construction: preserve how to recognize a valid result, as well as how to produce a candidate. A useful skill helps the executor decide when its procedure applies, what must be adapted, and which checks must hold before accepting the output. The feedback-based account connects this target to the methods' construction procedures as an interpretation of the observed patterns.

Evaluation should correspondingly report paired source-task utility, examine the same frozen artifact under distinct task changes, and inspect cases where the outcomes diverge. The three probes describe a profile of applicability: passing one change does not substitute for checking another. Reporting that profile alongside improvement and regression counts exposes behavior that a single average hides. It also makes the learning question concrete: which decisions remain useful when the source task no longer supplies the same inputs or demands the same output?

6. RELATED WORK

Learning persistent skills from experience. SkillEvolver, SPARK, and CoEvoSkills make experience persistent through different construction loops: deployment-grounded revision, verified-trajectory distillation, and co-evolutionary verification, respectively [1–3]. These methods motivate our focus on the artifact delivered to a later executor. We study how that artifact's usefulness changes after its source task is varied, and interpret the resulting behavior through its retained guidance.

Evaluating skill utility and use. SkillsBench evaluates skill utility across diverse tasks, while SWE-Skills-Bench studies skill use in software engineering [4, 5]. SkillLearnBench evaluates continual skill generation on real-world tasks [6]. From Raw Experience to Skill Consumption separates experience generation, skill extraction, and downstream consumption [7]. Our study complements these evaluations by connecting repeated source-task outcomes with controlled variations of the task and skill-level examples. The central object is the scope of usefulness of a frozen skill, rather than its score on the source task alone.

7. CONCLUSION

We examine what self-evolving agent skills preserve through repeated standard-task evaluations, task variation, and inspection of skill reuse. SkillEvolver, SPARK, and CoEvoSkills all yield positive source-task gains in both runs, but their variant results differ substantially. SkillEvolver has the broadest observed generalization profile, remaining positive under representation, instance, and requirement changes. SPARK's benefit depends on the change type, while CoEvoSkills' averages become negative. The inspected examples associate successful reuse with guidance for selecting valid results and preserving task-specific correctness rules. Together, these findings show why diagnosing skill learning requires examining how a frozen artifact guides behavior after the task changes.

8. REFERENCES

[1] Genrui Zhang, Erle Zhu, Jinfeng Zhou, Caiyan Jia, and Hongning Wang, "SkillEvolver: Skill Learning as a Meta-Skill," 2026.

[2] Yang Zhou, Zihan Dong, Zhenting Wang, Can Jin, Shiyu Zhao, Bangwei Guo, Difei Gu, Linjun Zhang, Mu Zhou, and Dimitris N. Metaxas, "Evidence Over Plans: Online Trajectory Verification for Skill Distillation," 2026.

[3] Hanrong Zhang, Shicheng Fan, Henry Peng Zou, Yankai Chen, Zhenting Wang, Jiayu Zhou, Chengze Li, Wei-Chieh Huang, Yifei Yao, Kening Zheng, Xue Liu, Xiaoxiao Li, and Philip S. Yu, "CoEvoSkills: Self-Evolving Agent Skills via Co-Evolutionary Verification," 2026.

[4] Xiangyi Li, Yimin Liu, Wenbo Chen, Bingran You, Zonglin Di, Yifeng He, Shenghan Zheng, Kyoung Whan Choe, Jiankai Sun, Shuyi Wang, Chujun Tao, Binxu Li, Xuandong Zhao, Hejia Geng, Xiaojun Wu, Junwei Zhou, Xiaokun Chen, Hanwen Xing, Yubo Li, Qunhong Zeng, Di Wang, Yuanli Wang, Roey Ben Chaim, Penghao Jiang, Haotian Shen, Luyang Kong, Xinyi Liu, Runhui Wang, Xuanqing Liu, Jiachen Li, Xin Lan, Yueqian Lin, Wengao Ye, Junwei He, Songlin Li, Yue Zhang, Yipeng Gao, Yijiang Li, Ze Ma, Liqiang Jing, Tianyu Wang, Kaixin Li, Yiqi Xue, Haoran Lyu, Yizhuo He, Yuchen Tian, Shutong Wu, Bowei Wang, Yixuan Gao, Bo Chen, Litong Liu, Sikai Cheng, Jiajun Bao, Shuaicheng Tong, Shuwen Xu, Terry Yue Zhuo, Tinghan Ye, Qi Qi, Miao Li, Longtai Liao, Zelin Tan, Chang Shi, Xilin Tang, Srinath Tankasala, Boqin Yuan, Yaoyao Qian, Jianhong Tu, Chenguang Wang, Yizhou Sun, Wei Wang, Aaron Taylor, Ziyue Yang, Changkun Guan, Zhikang Dong, Xinyu Zhang, Steven Dillmann, Han chung Lee, and Dawn Song, "SkillsBench: Benchmarking How Well Agent Skills Work Across Diverse Tasks," 2026.

[5] Tingxu Han, Yi Zhang, Wei Song, Chunrong Fang, Zhenyu Chen, Youcheng Sun, and Lijie Hu, "SWE-Skills-Bench: Do Agent Skills Actually Help in Real-World Software Engineering?," 2026.

[6] Shanshan Zhong, Yi Lu, Jingjie Ning, Yibing Wan, Lihan Feng, Yuyi Ao, Leonardo F. R. Ribeiro, Markus Dreyer, Sean Ammirati, and Chenyan Xiong, "SkillLearnBench: Benchmarking Continual Learning Methods for Agent Skill Generation on Real-World Tasks," 2026.

[7] Zisu Huang, Jingwen Xu, Yifan Yang, Ziyang Gong, Qihao Yang, Muzhao Tian, Xiaohua Wang, Changze Lv, Xuemei Gao, Qi Dai, Bei Liu, Kai Qiu, Xue Yang, Dongdong Chen, Xiaoqing Zheng, and Chong Luo, "From Raw Experience to Skill Consumption: A Systematic Study of Model-Generated Agent Skills," 2026.